

Day 12: Show Transactions Using a Loop

1. Day Number

Day 12

2. Topic Name

Loop Through a List

Today you will learn how to use a `for` loop to read and print every item stored inside a list.

3. Connection with Yesterday

Yesterday, you created a transaction list and added a deposit transaction using `append()`.

For example, your list could contain:

```
transactions = ["Deposit $500"]
```

Today, you will display the stored transactions **one by one** instead of printing the entire list at once.

4. Important Topics

- List
 - `for` loop
 - Loop variable
 - Empty-list check
 - `if-else`
 - Function
 - Printing list items individually
-

5. Foundational Notes

What is a list?

A list stores multiple values in one variable.

```
fruits = ["Apple", "Banana", "Mango"]
```

Here, `fruits` contains three strings.

What does a `for` loop do?

A `for` loop visits every item in a collection one at a time.

```
for fruit in fruits:  
    print(fruit)
```

The loop works like this:

```
First loop → fruit = "Apple"
Second loop → fruit = "Banana"
Third loop → fruit = "Mango"
```

What is an empty list?

A list with no items is called an empty list.

```
transactions = []
```

An empty list is considered False inside an if condition.

```
if transactions:
    print("The list has transactions.")
else:
    print("No transactions found.")
```

Why check whether the list is empty?

When there are no transactions, the loop has nothing to print.

Instead of displaying a blank output, the program should show a helpful message:

```
No transactions found.
```

6. Easy Example

This example prints menu options one by one:

```
def show_options(options):
    if options:
        for option in options:
            print(option)
    else:
        print("No options found.")
```

Example list:

```
menu_options = ["Check Balance", "Deposit Money", "Exit"]
```

Conceptually, the loop prints:

```
Check Balance
Deposit Money
Exit
```

This is only a simple example. It is not the complete solution to today's transaction problem.

7. Problem Statement

Create a list containing three transaction strings.

Example transaction ideas:

```
Deposit $500  
Withdrawal $100  
Deposit $200
```

Create a function that receives the transaction list.

Inside the function:

1. Check whether the list contains any transactions.
2. If the list is not empty, use a for loop.
3. Print every transaction one by one.
4. If the list is empty, print:

```
No transactions found.
```

8. Concepts Used

List

Stores multiple transaction strings.

```
transactions = [...]
```

Function

Keeps the transaction-display logic in one reusable place.

```
def show_transactions(...):
```

if-else

Checks whether transactions are available.

```
if transactions:  
    ...  
else:  
    ...
```

for loop

Reads each transaction from the list.

```
for transaction in transactions:  
    ...
```

Loop variable

The variable transaction temporarily stores the current list item.

For this list:

```
["Deposit $500", "Withdrawal $100", "Deposit $200"]
```

The loop variable changes like this:

```
transaction = "Deposit $500"  
transaction = "Withdrawal $100"  
transaction = "Deposit $200"
```

9. Thought Process

Think about the problem in small steps:

Step 1: Create the transaction list

Store three transaction messages inside one list.

```
transactions  
├── Deposit transaction  
├── Withdrawal transaction  
└── Deposit transaction
```

Step 2: Create a function

The function should accept the transaction list as an argument.

```
show_transactions(transaction_list)
```

Step 3: Check whether the list is empty

Before starting the loop, ask:

```
Does the list contain at least one item?
```

Step 4: Loop through the list

When transactions exist, visit each transaction one by one.

```
Take first transaction → print it  
Take second transaction → print it  
Take third transaction → print it
```

Step 5: Handle an empty list

When the list has no items, print a clear message instead of running the loop.

10. Pseudocode

```
START  
  
CREATE a list containing three transaction strings  
  
DEFINE a function that accepts a transaction list  
    IF the transaction list is not empty  
        FOR each transaction in the transaction list  
            PRINT the transaction  
    ELSE  
        PRINT "No transactions found."  
  
CALL the function and pass the transaction list  
  
END
```

11. Suggested Solving Approach: Functional Approach

Use a function such as:

```
show_transactions()
```

The function should be responsible only for displaying transactions.

Suggested structure:

```
Create transaction list  
  
Define show_transactions function  
    Check the list  
    Loop through transactions  
    Print each transaction  
    Handle empty list  
  
Call show_transactions
```

A functional approach is useful because the same function can display different transaction lists later.

```
show_transactions(today_transactions)  
show_transactions(yesterday_transactions)  
show_transactions(all_transactions)
```

12. Easy Edge Case

Empty transaction list

Example:

```
transactions = []
```

Because the list contains no items, the loop should not attempt to display transactions.

Expected message:

```
No transactions found.
```

Remember:

```
if transactions:
```

means:

```
If the list contains at least one item
```

13. Expected Input

There is **no user input** required in today's basic problem.

The transactions are already stored inside the program as a list.

Conceptual data:

```
Deposit $500  
Withdrawal $100  
Deposit $200
```

14. Expected Output

For a list containing three transactions:

```
Deposit $500  
Withdrawal $100  
Deposit $200
```

For an empty list:

```
No transactions found.
```

15. Hint Only

Create a function that accepts a transaction list. First use an `if` condition to check the list. Inside the `if` block, use:

```
for transaction in transactions:
```

Then print the loop variable. Use the `else` block for the empty-list message.